ROM REVERSE-ENGINEERING WORKBENCH

ROMFlow Studio User Manual

Interactive 68HC11 disassembly, program-flow maps, calibration discovery and portable project snapshots.

Version: 0.2 **Date:** November 2026 **Project:** snow663/7427HWContract

PURPOSE

ROMFlow Studio is a local browser workbench backed by a Python decoder. Use it to navigate decoded ROM instruction paths, annotate probable function boundaries, visualize links, examine memory/IO accesses, and define human-readable calibration tables. The program does not patch or execute the input ROM.

READ THIS FIRST

A decoded instruction or graph edge is not proof that the hardware actually executes it. Some ROM data happens to look like legal opcodes, and some indirect transfers cannot be proven statically. Keep byte-derived evidence, imported hints and personal hypotheses distinct.

Contents

1. Installation and first launch
2. Open a ROM and select an analysis profile
3. Tour of the interface
4. Routine tree and interactive program-flow graphs
5. Labels, definitions, colors and organization
6. Code/data regions and calibration tables
7. ROM/RAM, registers, stacks and SCI access tracing
8. Save, reopen and transfer a .romflow project
9. Practical reverse-engineering workflows
10. Keyboard shortcuts and tips
11. Troubleshooting, safety and known limits
12. Reference: files, evidence and terminology

1. Installation and first launch

- **Requirement:** Python 3.10 or newer. No extra pip packages are needed for the bundled application.
- **Windows:** open the repository folder **ROMFlow_Studio** and double-click **Launch_ROMFlow_Studio.cmd**.
- **Linux/macOS:** run **./Launch_ROMFlow_Studio.sh**, or launch from the directory with **python -m studio.server**.
- Open **http://127.0.0.1:8765/** in a browser if it is not opened automatically. The server listens on the local computer; it is not hosted publicly.

Keep the terminal or launcher running while using the application. Closing the browser does not remove saved workspace metadata; closing the backend ends the live session until it is restarted.

First-run checklist

- Confirm ROMFlow Studio loads and the Explorer and graph area appear.
- Select Generic HC11 for an unknown controller.
- Click Open ROM and select a raw .bin, .rom or .img file.
- Find the reset/interrupt roots and review uncertain regions.
- Save an explicit .romflow snapshot before making substantial manual classifications.

2. Open a ROM and select an analysis profile

Click **Open ROM...** and choose the ROM binary. ROMFlow accepts images up to 64 KiB. A full 64-KiB dump maps to address $0000; a smaller dump is normally top-aligned in the 64-KiB CPU space (for example, 32 KiB at $8000). Confirm the load address for your image before trusting vector or instruction addresses.

The **Generic HC11** profile avoids assuming GM-specific RAM and I/O locations. The **GM 7427 / $31** profile supplies memory-domain labels and optional calibration candidates. It is an *annotation profile*, not a different opcode decoder. Do not apply $31 assumptions as verified facts to other controller families.

What to validate immediately

- Check the ROM hash and image size in the header. Different calibration revisions deserve different saved projects.
- Inspect the vector entries, reset landing and recognizable initialization instructions.
- Distinguish blank fill, unmapped windows and genuine executable bytes; an 0xFF byte can itself be a legal opcode at an instruction boundary.
- If the ROM includes banked code, treat the visible image and CPU address mapping as a hypothesis until the bank-select logic is understood.

3. Tour of the interface

Top bar	ROM name, hash, selected hardware profile, calibration hints, Recent, Reports, Save project, Open project and Open ROM.
Explorer - left	Expandable groups for routines, vectors, dispatch edges, memory zones, annotated code/data regions and calibration tables.
Control-flow canvas - center	Routine or Program graph. Inspect branch and call relationships; zoom, pan and move flow nodes.
Bottom tabs	Disassembly, Memory & I/O, ROM Hex, Calibration and ROM Reads.
Inspector - right	Edit selected routine, instruction or address labels; define flags, semantic notes and interpreted math; view cross-references.
Status bar	Current analysis status and local/offline indication.

4. Routine tree and interactive flow maps

Use the Explorer caret to expand or collapse categories. The search field filters routines, addresses and labels. **Expand** and **Collapse** act on the tree. Click a routine to open its graph and disassembly.

Routine mode versus Program mode

- **Routine:** focuses on blocks, local exits, conditional paths and calls associated with the selected entry point.
- **Program:** navigates relationships among discovered routines and dispatch landing entries; use this to find high-level structure.
- Click a node or edge to open supporting information in the Inspector. On a selected edge, use Inspect call site or Inspect target to navigate.
- Drag nodes to make a readable layout; drag the background to pan; scroll or use +/- to zoom; Fit resets framing. Positions and transforms are preserved in project metadata.

The graph legend distinguishes ordinary flow, calls, conditional edges and unresolved transfers. An automatically inferred dispatch edge is not proof of runtime selection; verify pointer bytes and control paths independently.

Adding a missing entry point

Click **+ Entry**. Supply a hex address for an additional root. You may also specify a suspected dispatch-table location and slot count. This triggers a fresh code scan, but a declared table is **user evidence**, not proof that bytes are executable pointers.

5. Labels, definitions, colors and organization

Click a routine, instruction, or memory address. In the Inspector, edit Symbol/Label, Function group, Color, Category/semantics, Definition/notes, Flag bits, and Derived math/interpretation; click **Save annotation**. Existing names are an overlay and never rewrite the underlying ROM.

Recommended naming scheme

```
R_7A4F_major_dispatch
R_77C7_timer_compare_housekeeping
RAM_0250_injector_pw_candidate
ASIC_3FCE_fuel_handoff_candidate
CAL_4A00_VE_candidate
```

Use functional groups such as **Boot, Timer/ISR, Dispatcher, Fuel, Spark, Idle/IAC, ALDL/SCI, Diagnostics, Calibration, Unknown**. Keep provisional labels explicitly marked **candidate** until supported by a byte trace or hardware observation.

Color conventions you can adopt

Cyan	Verified instruction/code region or reviewed control path
Purple	Call relationships, reusable math helpers
Amber	Conditional path, speculative table, undecided semantics
Red	Unresolved indirect control flow or a review blocker
Muted/gray	Unknown, data not yet classified

These are suggested labeling conventions; the actual interface palette is editable. Colors organize the investigation, not the confidence calculation.

6. Code/data regions and calibration tables

Mark code, data or embedded pointer regions

Click **+ Region**. Enter the first and last addresses, classification, label, group and notes. Such regions mark your interpretation of bytes: the UI does not modify them. A mistaken region definition should be corrected rather than taken as decoder truth.

Define a table

Click **+ Table** or choose **Define table** at a memory address. Provide starting address, rows, columns, element width (bytes), signedness, endianness, scale, bias/offset, engineering units, row and column captions, flag-bit definitions and notes. The Calibration tab displays a read-only interpretation.

```
Engineering value = raw_value * scale + offset
```

```
Example: raw = 128, scale = 0.390625, offset = 0
         result = 50.0 percent
```

That example is a mathematical conversion only; it is **not** proof that a particular calibration uses that encoding. For flags, document the bit position and condition separately, such as **7=Output enabled**, only after the containing register or variable has been verified.

Table-discovery tips

- First locate code references to the suspected ROM table. Determine whether instructions read bytes or words, and whether indexed addresses are bounded.
- Inspect nearby table headers, dimensions and strides before setting rows and columns.
- Check big-endian versus little-endian values, signedness and fixed-point scaling against known observed quantities.
- If a pointer table is inside a code-containing ROM range, annotate it as embedded data and verify all landing targets individually.
- Use **Calibration hints** only with the $31 profile; imported tables are candidates and must be compared to the loaded ROM revision.

7. Memory, registers, stack and SCI tracing

Select **Memory & I/O** to view resolved memory accesses and their addresses, types and classifications. The **ROM Reads** tab aggregates code-origin reads of ROM addresses. Clicking an address opens read/write cross-references in the Inspector. ROM Hex shows the raw bytes surrounding an address.

What access evidence means

R	Instruction explicitly reads an address or a conservatively resolved indexed target.
W	Instruction stores or writes a target.
RW	Read-modify-write instruction; may touch individual bits.
Unresolved indexed	Register-base or path uncertainty prevents a trustworthy effective address.
Control reference	Branch/call/pointer relationship, not a normal RAM data access.

Do not force an effective address from a stale preceding LDX/LDY in another path. If a register is reloaded, used as a stack pointer, modified by ABX or affected by a call, resolution may be incomplete.

SCI / ALDL workflow

- Create an **ALDL/SCI** group and annotate the SCI control/data registers for the correct HC11 INIT mapping.
- Use the memory address Inspector to collect all reads of status/data and writes to transmit/control registers.

- Inspect caller routines and interrupt sources; separately mark the RX entry, TX queue, framing state and error path.
- Note that physical serial side effects, data-register read-to-clear order and interrupt timing require hardware-specific validation.

Stack and register caution

The application decodes explicit memory operands and shows local derived math. It does not yet offer a complete stack-frame reconstruction or a proven SSA data-flow engine. PSHA/PULX, JSR/RTS, SWI/RTI and SP-relative effects may require tracing beyond the memory-access inventory. Record suspected stack effects in the Inspector until independently checked.

8. Saving, reopening and moving a project

Automatic workspace (same computer)

ROMFlow automatically persists ROM-specific metadata under **workspace/projects/<ROM SHA prefix>/annotations.json**. Use **Recent** to reopen a cached ROM. This is a convenience cache, not a portable backup and not a Git-tracked research record.

Portable snapshot (.romflow)

Press **Save project** or **Ctrl+S** to download a self-contained **.romflow** file. Move it to another computer and choose **Open project...** or **Ctrl+O**. The source ROM is embedded, so you do not need to locate the original BIN again.

```
Example: BMHM_31_fuel_spark_review_2026-11-03.romflow
Archive members:
  manifest.json  format, ROM name, size, SHA-256, load address
  rom.bin        exact original input bytes
  metadata.json  annotations, graph layout, regions and tables
```

The loader verifies the ROM size, SHA-256 digest and project schema before replacing the active workspace. Static analysis is recalculated when the project is opened; annotation and view state are reapplied.

Saving versus exporting reports

Reports (Ctrl+E) exports analysis artifacts for inspection and sharing. These are not equivalent to a reopenable project snapshot. For full preservation, save a **.romflow** file. For a human-readable investigation record, save the report bundle as well.

For reliable versions, keep milestone snapshots outside the workspace cache, such as 01_initial_rom, 02_vectors_reviewed, 03_dispatch_verified, and 04_fuel_path_traced.

9. Practical workflows

Workflow A - Find the reset path on a new PCM

1. Open a ROM under Generic HC11 and check its address mapping.

2. Follow reset and other vector targets; inspect immediate initialization writes.

3. Mark expected register relocation and timer setup as candidate code.

4. Name each reviewed routine and tag the execution-context evidence.

5. Export reports and preserve a .romflow snapshot.

Workflow B - Reconstruct a dispatch scheduler

1. Locate computed JMP/JSR instructions or pointer-table consumers.

2. Inspect the index calculation and word/byte offsets.

3. Define the candidate dispatch table with the correct width and count.

4. Compare each landing target to actual ROM instructions and the graph.

5. Annotate the call site, table and each slot; keep unproven edges marked inferred.

Workflow C - Follow fuel, spark or IAC output ownership

1. Select a suspected timer or ASIC handoff address in Memory & I/O.

2. Inspect every decoded writer, reader and relevant bit-modifying instruction.

3. Follow upstream calculations to RAM variables and calibration reads.

4. Record math, integer width, sign, carry conditions and units in each routine.

5. Check write order and physical effects separately against real logs or bench traces.

Workflow D - Compare a similar GM PCM

1. Save the completed reference project; open the second PCM separately.

2. Start Generic HC11; do not transplant the first PCM memory layout.

3. Find equivalent vectors, timer patterns, call structure and table consumers.

4. Use groups and labels to document matched, changed and missing behavior.

5. Treat hardware-address differences as new contract evidence, not just relocation.

10. Shortcuts and tips

Save portable project	Ctrl+S (Cmd+S on supported browsers)
Open portable project	Ctrl+O (Cmd+O on supported browsers)
Open a raw ROM	Ctrl+Shift+O
Export reports	Ctrl+E
Close Inspector overlay	Esc
Fit flow graph	Fit button
Zoom graph	Mouse wheel over graph or + / - buttons
Move graph	Drag background to pan; drag nodes to arrange
Expand / collapse explorer	Buttons at top of Explorer

Tricks for large investigations

- Work from high-confidence roots outward; avoid a linear 64-KiB disassembly as the primary evidence source.
- Use distinctive names for routine entry, code address, and RAM/ASIC symbol to avoid conflating the concepts.
- Document all exit conditions, not only the normal return. Interrupts and tail jumps can share instruction bodies.
- Save graph layouts after reorganizing major subgraphs; restoring a snapshot also restores your visual arrangement.
- Give uncertainty its own group so unresolved branches or suspicious tables remain visible.
- If the global graph is crowded, switch to Routine mode; the renderer limits very large global views for responsiveness.

11. Troubleshooting and current limitations

No page at localhost	Confirm Python 3.10+, launcher remains running and port 8765 is reachable locally.
Wrong addresses or no reset roots	Verify ROM length, CPU mapping/load address, vectors and whether data is banked.
Routine missing	Add a verified root using + Entry; record why this address is a valid entry.
Unexpected calibration values	Review width, byte order, signedness, rows/columns, scale and offset.
Flow line unresolved	Indirect branch, table consumer or register base may require explicit evidence.
Lost work after switching machines	Use Save project; automatic workspace cache stays with the original computer.
Project rejected as corrupt	Check that it is an unmodified .romflow archive with ROM hash and manifest intact.
Huge graph too slow	Use a single routine instead of the full Program graph.

Known limits in v0.2: static control-flow is conservative; code/data boundaries can remain disputed; dynamic bank mapping, complete hardware effect semantics and detailed global symbolic execution are not proven. Tables are views rather than active editors, and the tool does not write a modified ROM. Synthetic regression fixtures are not substitutes for a verified real factory-ROM comparison.

Engine-control caution: before modifying ignition or injector output software, independently validate timing, watchdog behavior, interrupt ownership, write ordering, fail-safe states and ASIC latch semantics. This GUI is a research instrument, not proof of safe engine operation.

12. File and terminology reference

studio/	HC11 decoding, ROM analysis, graph and math-trace support, local HTTP server.
static/	Browser UI assets (HTML, CSS, JavaScript).
profiles/	Optional GM memory/annotation hints.
examples/	Synthetic ROM fixtures for regression tests.
tests/	Unit tests covering graphs, tables and project round trips.
workspace/	Local autosave cache; excluded from Git.
docs/	This user manual and future reference documentation.

Evidence terminology

- **Rooted decode:** instruction stream followed from a known/provided entry and branch/call successors.
- **Direct edge:** control transfer target encoded by the instruction bytes.
- **Inferred dispatch:** possible target reconstructed from code/table interpretation; runtime selection not proven.
- **Annotation:** user-entered meaning, function group, register name, calibration interpretation or hypothesis.
- **ROM fingerprint/hash:** SHA-256 of exact source bytes, used to associate analysis and verify portable project integrity.

For developers: run **python -m unittest discover -s tests -v** from the ROMFlow_Studio root. Review failures before updating the main branch. This manual documents the current v0.2 interface and is not a promise that every future analysis feature is already implemented.